

PVD Instrument Verification

How the Juniper Test Station verifies that the Vitrek V71 itself is measuring correctly, using a Vitrek **APVD-74** Performance Verification Device.

Many product safety agencies require an electrical safety tester to undergo a daily verification of performance. This is that check — and unlike the instrument's own routine, it records the numbers.

Why the software does this rather than driving the V71's built-in routine

The V71 front panel has `UTILITY MENU → AUTO PVD`, which walks an operator through Vitrek's own verification against an APVD. It is a good routine. It is also **front-panel only**.

Section 6 of the *V7X Series Operating Manual* lists the complete remote command set for USB and RS-232. There is no command for AUTO PVD, and none for SELF TEST or CAL VERIFY either.

`RUN` starts the *active programmed sequence* and nothing else. There is no way to ask the instrument to run its own verification over the interface.

So the station reproduces the verification as an ordinary programmed sequence against the APVD's loads. The manual explicitly sanctions this — Section 7 notes that "the user may wish to not use this built-in sequence but use their own sequence."

This turns out to be the better artefact for a production floor:

	Built-in AUTO PVD	This implementation
Result	PASS/FAIL on the front panel	Measured value for every point
Record	Operator writes it down	Stored in SQLite, exported to Excel
Trend	None	Every historical reading is queryable
Tolerance	Fixed in firmware	Explicit, visible, adjustable per point

What the V71 can and cannot verify

The V71 is an ACW / DCW / CONT instrument. It has no insulation-resistance mode and no ground-bond mode (manual, *Available Models*: "V71. ACW, DCW and CONT modes").

The APVD-74 carries hipot, low-resistance **and** ground-bond loads. On a V71, the ground-bond and IR verification points therefore cannot be exercised. They are not an error and they are not silently dropped — the runner records them with verdict `SKIPPED` and the reason `GB not supported by V71`, so the verification record shows the full profile and exactly which points the instrument was incapable of.

Pushing an unsupported step at the V71 would earn `ERR 2` and abort the whole sequence build, which is why capability gating happens before programming rather than after.

If the station ever moves to a V74 or V77, those points start running with no configuration change.

The two gates a point must clear

A point passes only if **both** of these hold:

1. **Instrument integrity.** `STEPRSLT?` returns a status-flag bitmask. Anything non-zero — DUT breakdown, arc detected, interlock failure, user abort, over-voltage, overheating — fails the point outright.
2. **Measurement window.** The measured value must fall inside `nominal ± tolerance`.

The order matters. A step that arced can still hand back a plausible-looking number, and treating that as a passing measurement would be the worst thing this feature could do.

Note what the step limits sent to the V71 are *for*: they are deliberately wide, so the instrument runs the step to completion and gives us a reading instead of tripping out. **The instrument's own pass/fail is not the verdict.** The verdict is computed here.

Baselining — required before the first real verification

Vitretek does not publish the internal load values of the APVD devices. Every point in `pvd_profiles.json` therefore ships with `nominal: null`.

While a profile has any point without a nominal, the software **will not issue a PASS**. The run returns `BASELINE` instead. An unbaselined profile that could report a passing verification would be worse than no verification at all.

To baseline:

1. Confirm the V71 is **inside its calibration period**. Everything downstream inherits the accuracy of the instrument you baseline against.
2. Run a `SELF TEST` from the V71's Utility Menu first (recommended by the manual before any verification work).
3. If you will use the CONT point, run `CONT ZERO` with the leads shorted so lead resistance is offset out. See Appendix A of the operating manual.
4. On the **PVD Verify** page, choose the `APVD-74` profile, set mode to **Baseline**, and run it.

5. Review the captured values. They should look physically sensible for the loads involved.
6. Press **Promote to Nominals**, choosing a tolerance. This writes the measurements into `pvd_profiles.json` as the nominals, along with which run and which instrument they came from.

From then on, **Verify** mode issues real PASS/FAIL verdicts.

Choosing a tolerance

The default is $\pm 10\%$, which is deliberately loose for a daily go/no-go check. The window has to cover:

- the V7X's own measurement accuracy at that range,
- the APVD load's tolerance,
- normal line-voltage and temperature variation.

Tighten it only after several baseline runs give you a feel for the real spread. A window tighter than the instrument's own accuracy generates false failures, and a verification that cries wolf gets ignored — which is the actual risk.

For any quantity that can legitimately sit near zero, use `tol_abs` rather than `tol_pct`; a percentage window around a near-zero nominal collapses to nothing.

Running a verification

Verification is **admin-only**. Running one decides whether the instrument is fit to certify product, so it is a controlled activity rather than a shift task. The verification history is visible to everyone.

1. Connect the V71 on the **HiPot** page.
2. Go to **PVD Verify**, log in as admin, pick the profile, set mode to **Verify**, enter the operator name and the APVD serial.
3. Press **Start Verification**.
4. The station shows a connection prompt for each phase, matching what the V71's own AUTO PVD screens ask for. Make the connections, then press **Connections Made — Continue**.
5. Results appear per point as they complete, with the measured value, the nominal, and the deviation.

Phases

Each phase runs as its **own programmed sequence**, not as one sequence with `HOLD` steps in it. That is deliberate: between phases the instrument is idle and de-energised while hands are on the leads. (`add_hold_step()` exists in the driver and works, if a future sequence genuinely needs a mid-run prompt.)

Phase	Connections	Points on a V71
1 — Hipot load	White plug → HV terminal; red & black plugs → CONT terminals; ring lugs disconnected	ACW 1000 V, DCW 1000 V
2 — Low resistance load	Red & black ring lugs joined across the APVD low-resistance load	CONT
3 — Ground bond load	K-2R leads to the APVD-74 GB terminals	<i>(skipped — V71 has no GB)</i>

⚠ Phase 1 applies high voltage. Confirm the enclosure is closed and nobody is in contact with the DUT area before continuing.

Test voltages in the shipped profile are conservative defaults (1000 V). Confirm the APVD-74's voltage ratings with Vitrek before raising them.

Where results go

Store	What
pvd_runs	One row per verification: profile, mode, operator, V71 model/serial/firmware, APVD serial, overall result
pvd_points	One row per point: measured, nominal, window, deviation %, status flags, verdict, reason
Excel export	PVD Verification sheet in the standard workbook, ready for SharePoint

`passed` is deliberately `NULL` — not `0` — for `BASELINE` and `ABORTED` runs. Those produce measurements but no verdict, and recording them as failures would make the verification history lie.

API

Method	Endpoint	Role	Description
GET	/api/pvd/profiles	any	List profiles and whether each is baselined
GET	/api/pvd/profile/<id>	any	Full profile: phases, points, tolerances
POST	/api/pvd/start	admin	Start a run (profile_id , mode , operator , pvd_serial , simulate)
GET	/api/pvd/status	any	Poll state, current connection prompt, live results
POST	/api/pvd/ack	admin	Confirm the phase connections are made
POST	/api/pvd/stop	admin	Abort and make the instrument safe
GET	/api/pvd/results	any	Verification history
GET	/api/pvd/result/<id>	any	One run with all its points
GET	/api/pvd/verification_status	any	Last passing run and whether it is still current
POST	/api/pvd/baseline/promote	admin	Adopt a baseline run's measurements as nominals

`simulate: true` runs the whole flow — phases, prompts, evaluation, storage — with synthetic measurements and no instrument. Useful for training an operator on the sequence without energising anything.

Verification currency

`/api/pvd/verification_status` reports when the instrument last passed and whether that is inside the profile's `recommended_interval_hours` (24 by default). The PVD page shows this as *current* or *OVERDUE*.

Nothing in this codebase blocks a production test on an overdue verification. That gate belongs to the quality procedure, not to the app — and wiring it in without that decision being made deliberately would be the app quietly inventing policy. Say the word and it becomes a hard stop.

Files

File	Role
pvd_profiles.json	Verification profiles: phases, points, nominals, tolerances
pvd_test.py	Runner, tolerance evaluation, baseline promotion, currency
database.py	pvd_runs / pvd_points schema and CRUD
app.py	/api/pvd/* routes and the PVD Verify page
excel_export.py	PVD Verification sheet